7.17.TRAVELLY NBP DATABASE DESIGN v1.0

Final 20-Table Architecture for the One-Month Working Release

Project: Travelly

Prepared for: S. Kajarsan

Architecture status: FROZEN IMPLEMENTATION BASELINE

Document date: 21 October 2026

Database target: PostgreSQL 16 with PostGIS enabled for geographic queries

Supersedes: Travelly Database Design v2.0 — Final 149-Table Architecture

Important: the previous 149-table architecture and the revised proposal have been archived separately as PDFs. This document is the current implementation authority for the one-month NBP.

1. Purpose, authority and scope

This document defines the database that will be implemented for Travelly's first complete working release. It converts the wider long-term product vision into exactly 20 application tables while preserving the complete Passenger–Owner–Driver–Admin journey. The design supports private vehicle hire, basic vehicle rental, route and airport Ticket Hire with a 2D seat map, protected fulfilment, payments, Driver operations, maintenance records, notifications, basic Travelly Reels, road alerts and AI-assisted text/voice search.

The database is deliberately normalized only where separation protects data integrity, concurrency or permissions. Closely related supporting records are consolidated through typed rows and JSONB metadata so the one-month build remains achievable. A new feature may not introduce a twenty-first application table during the NBP unless another table is formally merged or removed through a documented migration decision.

This is a logical and implementation-ready design. API routes are documented separately after this schema is approved. Database files, migrations and production data are not created by this document.

1.1 Included product capabilities

• One shared account may hold Passenger, Owner and Driver roles; Admin is separately controlled.

• Private hire: one-way, return/up-and-down, airport, local, event, long-distance and All-Ceylon.

• Basic vehicle rental using the same vehicle, service, booking, payment and document records.

• Ticket Hire for approved routes and airports with published departures and selectable 2D seats.

• Owner self-drive inside the Owner workspace after Driver eligibility checks; no role-switching screen is required.

• External Driver assignment, acceptance, trip start, live trip status and completion.

• Travelly Protected Ticket fulfilment with an internal minimum-seat rule and Admin-assisted replacement.

• Online Ticket booking requires full online payment before seat confirmation. Cash is limited to enabled private-hire/basic-rental services with Owner-selected start/end collection timing, two-sided verification, refund/reallocation and post-trip Owner payable states.

• Vehicle documents, odometer, oil, tyre, alignment, service, fuel, charging and personal-use records.

• In-app, email, message and automated IVR Press 1/Press 2 confirmation outcomes.

• AI-assisted text/voice trip search that produces normal validated search filters.

• Basic vertical Reels, linked vehicle/route/departure discovery and core interactions.

• Active-trip, route-aware road alerts with expiry and moderation.

1.2 Explicit Phase 2 exclusions

• Subscription plans, subscription billing and secondary-owner delegation.

• Conversational AI phone calling.

• Full chat and direct-messaging history.

• Automatic cross-Owner financial settlement and a complete double-entry finance ledger.

• Route-segment seat resale; the NBP reserves a selected seat for the whole departure.

• Full GPS history and tracking outside an active platform trip.

• Advanced rental handover, damage, deposit and inspection workflow.

• Advanced social recommendation models, stories, live streaming and monetisation.

• Predictive diagnostics, charging-network integration and fully automatic maintenance diagnosis.

• A fully dynamic Admin configuration engine for every business rule.

2. Architecture principles

• PostgreSQL is the source of truth. UUID primary keys are used for externally exposed entities.

• Money uses NUMERIC(12,2) and ISO currency code; never use floating-point types for money.

• All timestamps use TIMESTAMPTZ and are stored in UTC. The application displays Asia/Colombo by default.

• Files, images and videos are stored in protected object storage/CDN. The database stores storage keys, URLs and metadata only.

• Booking, payment and trip states remain separate because each has different legal and operational meaning.

• Historical financial, seat and completed-trip facts are never silently overwritten.

• JSONB is allowed for bounded, low-query configuration such as luggage details, media arrays and accepted replacement limits. Core relationships remain foreign keys.

• Soft deletion is used only where retention is required. Immutable or completed business records are archived by status rather than deleted.

• Every sensitive read/write is authorised by active role, ownership and assignment; possession of an ID is never sufficient.

2.1 Common columns

Where appropriate, tables contain: id UUID PRIMARY KEY, created_at TIMESTAMPTZ NOT NULL DEFAULT now(), updated_at TIMESTAMPTZ NOT NULL DEFAULT now(), created_by UUID NULL REFERENCES users(id), updated_by UUID NULL REFERENCES users(id), version INTEGER NOT NULL DEFAULT 1 for optimistic concurrency, and metadata JSONB NOT NULL DEFAULT '{}'. deleted_at is used only for profiles/catalogue rows that may be soft-deleted. Public references such as booking_reference and trip_reference are unique, human-readable values generated by the application.

3. Final table catalogue

• 01 users — shared identity, authentication and common profile

• 02 user_roles — multiple role memberships and independent approval states

• 03 documents — user or vehicle document metadata and verification

• 04 vehicles — Owner fleet, capacity, luggage, power type, media and current odometer

• 05 service_types — Admin-controlled hire, ticket and rental service definitions

• 06 vehicle_services — services and pricing enabled for each vehicle

• 07 routes — approved travel corridors

• 08 route_stops — ordered pickup and drop-off points

• 09 scheduled_trips — published Ticket departures and protected-fulfilment snapshot

• 10 vehicle_seats — versioned 2D vehicle seat layout

• 11 scheduled_trip_seats — departure-specific seat inventory and temporary holds

• 12 bookings — unified private-hire, Ticket and rental booking header

• 13 booking_travellers — traveller details, seats and boarding/drop stops

• 14 trips — operator assignment, trip execution, odometer and current live location

• 15 payments — payment, reallocation, refund, charge and Owner payout transactions

• 16 vehicle_records — maintenance, expenses, odometer, personal use and availability blocks

• 17 notifications — delivery, read state and IVR response outcome

• 18 social_posts — Reels/photos/reviews with optional booking-discovery links

• 19 social_interactions — like, comment, save, share, follow, report and rating

• 20 road_alerts — geolocated, expiring and moderated route alerts

4. Detailed table specifications

4.1 users

Purpose: one identity and common profile for every person. A person is not duplicated when gaining another role.

Core columns:

• id UUID PK; public_user_reference VARCHAR(20) UNIQUE NOT NULL.

• email CITEXT UNIQUE NULL; phone_e164 VARCHAR(20) UNIQUE NULL; at least one verified contact is required before booking.

• password_hash TEXT NOT NULL for local authentication; external_auth_provider and external_auth_subject are nullable.

• full_name, display_name, profile_image_url, date_of_birth NULL, preferred_language, timezone.

• account_status: PENDING_VERIFICATION, ACTIVE, SUSPENDED, BLOCKED, CLOSED.

• email_verified_at, phone_verified_at, last_login_at, failed_login_count, locked_until.

• common_profile JSONB for non-authoritative presentation fields only.

Constraints and indexes: unique lower-case email, unique normalized phone, CHECK that email or phone exists; indexes on account_status and created_at. Password hashes, reset tokens and verification secrets must never be returned by ordinary profile queries.

4.2 user_roles

Purpose: allow one user to hold Passenger, Owner, Driver and Admin roles without duplicate accounts.

Core columns:

• id UUID PK; user_id UUID FK users(id) ON DELETE RESTRICT.

• role_code: PASSENGER, OWNER, DRIVER, ADMIN.

• approval_status: REQUESTED, UNDER_REVIEW, APPROVED, REJECTED, SUSPENDED.

• requested_at, approved_at, approved_by UUID FK users(id), rejection_reason, suspended_at, suspension_reason.

• role_profile JSONB for small role-specific preferences that do not justify another NBP table.

Rules: UNIQUE(user_id, role_code). Passenger may be automatically approved after account verification. Owner and Driver actions require APPROVED status. Admin role assignment is restricted to authorised administrators.

Owner self-drive: an Owner chooses Accept & Self Drive from the Owner dashboard. The same user must also have an approved DRIVER role and valid Driver documents. The application opens contextual Driver Mode inside the Owner workflow; the Owner does not need to log out or change dashboards.

4.3 documents

Purpose: a unified verification record for documents belonging to one user or one vehicle.

Core columns:

• id UUID PK; user_id UUID NULL FK users(id); vehicle_id UUID NULL FK vehicles(id).

• document_type VARCHAR(60): NIC, PASSPORT, DRIVING_LICENCE, BUSINESS_REGISTRATION, OWNER_AUTHORISATION, BANK_PROOF, VEHICLE_REGISTRATION, REVENUE_LICENCE, INSURANCE, EMISSION, FITNESS, ROUTE_PERMIT or Admin-supported type.

• document_number, issuing_country, issue_date, expiry_date.

• storage_key, secure_file_url, original_file_name, mime_type, file_size_bytes, file_checksum.

• status: DRAFT, SUBMITTED, UNDER_REVIEW, APPROVED, REJECTED, NEEDS_CORRECTION, EXPIRED, SUPERSEDED.

• submitted_at, verified_by UUID FK users(id), verified_at, rejection_reason, superseded_document_id UUID FK documents(id).

Integrity: CHECK ((user_id IS NOT NULL)::int + (vehicle_id IS NOT NULL)::int = 1). A superseded document is retained for history. Expiry queries use an index on (expiry_date, status). Document files use private storage and short-lived access URLs.

NBP requirements: Driver licence is mandatory. Individual Owner identity/licence and ownership/authorisation are checked under current business rules. Company fields are conditional. Normal Passenger booking does not require identity documents; basic rental may require licence and identity verification.

4.4 vehicles

Purpose: one profile for every Owner vehicle, including Passenger-visible details and operational state.

Core columns:

• id UUID PK; public_vehicle_reference VARCHAR(20) UNIQUE; owner_user_id UUID FK users(id); default_driver_user_id UUID NULL FK users(id).

• registration_number CITEXT UNIQUE NOT NULL; vehicle_type_code; make; model; manufacture_year; colour.

• power_type: ICE, MHEV, HEV, PHEV, BEV; engine_fuel_type: PETROL, DIESEL or NULL for BEV.

• seat_capacity SMALLINT; large_luggage_capacity SMALLINT; small_luggage_capacity SMALLINT.

• current_odometer_km NUMERIC(12,1); odometer_updated_at; transmission_type; air_conditioned BOOLEAN.

• description; media JSONB containing bounded inside/outside image objects and display order.

• verification_status: DRAFT, SUBMITTED, APPROVED, REJECTED, SUSPENDED.

• availability_status: INACTIVE, AVAILABLE, HELD, BOOKED, ON_TRIP, MAINTENANCE, BLOCKED.

• is_published BOOLEAN; approved_by; approved_at.

Rules: only an APPROVED Owner may publish an APPROVED vehicle. Capacity values must be non-negative. BEV requires engine_fuel_type NULL; ICE/MHEV/HEV/PHEV use applicable fuel data. Vehicle type is an Admin-supported passenger vehicle category and is not limited to bus and car.

Indexes: owner_user_id, verification_status/is_published, vehicle_type_code, availability_status and optional GIN index on selected searchable attributes.

4.5 service_types

Purpose: Admin-managed service catalogue without hard-coding every service rule.

Core columns:

• id UUID PK; code CITEXT UNIQUE; name; description.

• category: PRIVATE_HIRE, TICKET_HIRE, RENTAL.

• is_active BOOLEAN; display_order INTEGER.

• default_pricing_basis: FIXED, DISTANCE, TIME, PER_SEAT, OWNER_QUOTE.

• settings JSONB for bounded NBP values such as decision_lead_hours, booking_cutoff_minutes, hold_minutes, allowed_substitution_window_minutes, reconfirmation lead time and required document codes.

Seed examples: ONE_WAY, RETURN_UP_DOWN, AIRPORT_HIRE, LOCAL_HIRE, EVENT_HIRE, LONG_DISTANCE, ALL_CEYLON, AIRPORT_TICKET, ROUTE_TICKET and BASIC_RENTAL. Admin changes apply to new offers/departures; accepted booking terms are copied to the booking or scheduled trip.

4.6 vehicle_services

Purpose: many-to-many relationship between a vehicle and the services the Owner offers.

Core columns:

• id UUID PK; vehicle_id UUID FK vehicles(id); service_type_id UUID FK service_types(id).

• is_enabled BOOLEAN; pricing_basis; base_price NUMERIC(12,2); price_per_km; price_per_hour; currency CHAR(3).

• minimum_hire_hours, minimum_hire_km, available_from, available_until.

• owner_terms JSONB for permitted locations, luggage notes, rental-specific limits and quotation rules.

Constraints: UNIQUE(vehicle_id, service_type_id). Price values are non-negative. The service and vehicle must be active before appearing in Passenger search.

4.7 routes

Purpose: approved route definition shared by scheduled Ticket departures.

Core columns:

• id UUID PK; route_code CITEXT UNIQUE; route_name.

• origin_name, destination_name; origin_latitude, origin_longitude, destination_latitude, destination_longitude.

• estimated_distance_km, estimated_duration_minutes.

• status: DRAFT, ACTIVE, INACTIVE; created_by_owner_id NULL for Admin-owned routes; approved_by; approved_at.

• route_geometry/geography NULL when PostGIS is available; route_settings JSONB.

Indexes: origin/destination search, status, and GiST geography index when PostGIS is enabled.

4.8 route_stops

Purpose: ordered pickup/drop-off points for each route.

Core columns:

• id UUID PK; route_id UUID FK routes(id) ON DELETE CASCADE.

• stop_order SMALLINT; stop_code; stop_name; latitude; longitude.

• pickup_allowed BOOLEAN; dropoff_allowed BOOLEAN; scheduled_offset_minutes INTEGER; is_active BOOLEAN.

Constraints: UNIQUE(route_id, stop_order), UNIQUE(route_id, stop_code); valid coordinate ranges. The first and last stops represent route origin and destination.

4.9 scheduled_trips

Purpose: one published Ticket departure and the frozen Passenger promise for that departure.

Core columns:

• id UUID PK; departure_reference VARCHAR(24) UNIQUE; owner_user_id UUID FK users(id).

• service_type_id UUID FK service_types(id); route_id UUID FK routes(id).

• original_vehicle_id UUID FK vehicles(id); current_vehicle_id UUID FK vehicles(id).

• operator_driver_user_id UUID NULL FK users(id).

• departure_at, estimated_arrival_at, booking_cutoff_at.

• fare_per_seat NUMERIC(12,2); currency CHAR(3).

• fulfilment_policy: FIXED_GUARANTEED or MINIMUM_SEATS_WITH_PROTECTED_FULFILMENT.

• minimum_confirmed_seats NULL for fixed policy; decision_deadline; accepted_time_window_minutes; accepted_replacement_terms JSONB.

• status: DRAFT, PUBLISHED, BOOKING_OPEN, DECISION_REQUIRED, COMMITTED, READY, IN_PROGRESS, COMPLETED, CANCELLED, DISRUPTED.

• replacement_status: NOT_REQUIRED, OWNER_ALTERNATIVE, PLATFORM_SHORTLISTED, AWAITING_OTHER_OWNER, APPROVED, REJECTED, FAILED.

• replacement_scheduled_trip_id UUID NULL FK scheduled_trips(id); final_operating_owner_user_id UUID NULL FK users(id); committed_at; cancellation_reason.

Rules: the policy, deadline and substitution limits are copied when published. Later Admin changes do not rewrite accepted terms. If minimum seats are not met by the deadline, the Owner must run the original vehicle, use an eligible own vehicle, or use an Admin-assisted eligible replacement. No action by the deadline commits the original trip. Low seat count is not a valid post-deadline cancellation reason.

Replacement eligibility: valid vehicle/Driver documents, no time conflict, sufficient seats/luggage, required route/stops and equal or better accepted quality. A material route, fare, time or vehicle-standard change requires Passenger acceptance.

Indexes: (route_id, departure_at, status), (current_vehicle_id, departure_at), owner_user_id/status and decision_deadline/status.

4.10 vehicle_seats

Purpose: reusable, versioned 2D seat definition for a vehicle.

Core columns:

• id UUID PK; vehicle_id UUID FK vehicles(id).

• layout_version INTEGER; seat_code VARCHAR(12); row_number; column_number; deck_number DEFAULT 1.

• seat_type: STANDARD, WINDOW, AISLE, PREMIUM, ACCESSIBLE, CREW.

• is_active BOOLEAN; is_bookable BOOLEAN; display_metadata JSONB.

Constraints: UNIQUE(vehicle_id, layout_version, seat_code) and UNIQUE(vehicle_id, layout_version, deck_number, row_number, column_number). A published departure retains its created seat inventory even if a later layout version is introduced.

4.11 scheduled_trip_seats

Purpose: departure-specific inventory and concurrency boundary for each selectable seat.

Core columns:

• id UUID PK; scheduled_trip_id UUID FK scheduled_trips(id) ON DELETE CASCADE; vehicle_seat_id UUID FK vehicle_seats(id).

• seat_status: AVAILABLE, HELD, BOOKED, BLOCKED.

• held_by_booking_id UUID NULL FK bookings(id); hold_expires_at TIMESTAMPTZ NULL.

• booked_booking_id UUID NULL FK bookings(id); fare_override NUMERIC(12,2) NULL.

• version INTEGER NOT NULL DEFAULT 1.

Constraints: UNIQUE(scheduled_trip_id, vehicle_seat_id). HELD requires held_by_booking_id and hold_expires_at; BOOKED requires booked_booking_id. Seat hold and confirmation execute in one transaction using row locking or an atomic conditional update. An expired HELD row is safely returned to AVAILABLE. The NBP reserves one seat for the entire departure; segment reuse is Phase 2.

4.12 bookings

Purpose: one authoritative booking header for private hire, Ticket Hire or basic rental.

Core columns:

• id UUID PK; booking_reference VARCHAR(24) UNIQUE; passenger_user_id UUID FK users(id).

• booking_type: PRIVATE_HIRE, TICKET_HIRE, RENTAL; service_type_id UUID FK service_types(id).

• owner_user_id UUID NULL FK users(id); vehicle_id UUID NULL FK vehicles(id).

• original_scheduled_trip_id UUID NULL FK scheduled_trips(id); current_scheduled_trip_id UUID NULL FK scheduled_trips(id).

• origin_text, destination_text, origin_stop_id UUID NULL FK route_stops(id), destination_stop_id UUID NULL FK route_stops(id).

• start_at, end_at; passenger_count; large_luggage_count; small_luggage_count; special_requirements.

• status: DRAFT, HOLDING, REQUESTED, OWNER_ACCEPTED, DRIVER_ASSIGNMENT_PENDING, CONFIRMED, READY, IN_PROGRESS, COMPLETED, CANCELLED, REFUND_PENDING, REFUNDED, FAILED.

• assurance_type: STANDARD_HIRE, FIXED_DEPARTURE, TRAVELLY_PROTECTED; protected_terms_snapshot JSONB.

• quoted_amount, discount_amount, total_amount, currency; payment_status_snapshot.

• cancelled_by_user_id, cancelled_at, cancellation_reason, cancellation_policy_snapshot JSONB.

• idempotency_key VARCHAR(80) UNIQUE NULL; accepted_at; confirmed_at.

Rules: Ticket Passenger UI shows normal booking plus Fixed Departure or Travelly Protected assurance. It never exposes an uncertain ‘conditional booking’. A valid paid booking creates Travelly’s fulfilment obligation under the accepted terms. Original and current departure/vehicle links preserve replacement history inside the 20-table limit.

Indexes: passenger/status/created_at; owner/status/start_at; vehicle/start/end; scheduled trip/status; service/start.

4.13 booking_travellers

Purpose: people travelling under a booking, including each selected Ticket seat.

Core columns:

• id UUID PK; booking_id UUID FK bookings(id) ON DELETE RESTRICT.

• linked_user_id UUID NULL FK users(id); full_name; contact_phone; contact_email; age_category.

• is_primary BOOLEAN; identity_reference NULL where required by service.

• scheduled_trip_seat_id UUID NULL FK scheduled_trip_seats(id).

• boarding_stop_id UUID NULL FK route_stops(id); dropoff_stop_id UUID NULL FK route_stops(id).

• special_assistance; traveller_status: ACTIVE, MOVED, CANCELLED, BOARDED, COMPLETED.

Constraints: one primary traveller per booking; one active traveller per scheduled-trip seat; Ticket boarding stop must precede drop-off stop on the route.

4.14 trips

Purpose: operational execution after a booking/departure is accepted.

Core columns:

• id UUID PK; trip_reference VARCHAR(24) UNIQUE; booking_id UUID NULL FK bookings(id); scheduled_trip_id UUID NULL FK scheduled_trips(id).

• vehicle_id UUID FK vehicles(id); operating_owner_user_id UUID FK users(id); driver_user_id UUID NULL FK users(id).

• assignment_type: OWNER_SELF_DRIVE, EXTERNAL_DRIVER; assignment_status: PENDING, ACCEPTED, DECLINED, REASSIGN_REQUIRED, CANCELLED.

• status: PREPARING, READY, STARTED, PAUSED, BREAKDOWN, COMPLETED, CANCELLED.

• scheduled_start_at, scheduled_end_at, actual_start_at, actual_end_at.

• start_odometer_km, end_odometer_km, calculated_distance_km.

• tracking_status: OFF, ACTIVE, STALE, ENDED; current_latitude, current_longitude, location_accuracy_m, current_speed_kph, location_updated_at.

• breakdown_details, completion_notes.

Rules: Driver Online means available for requests and is not a live-location signal. Owner sees accurate location only when status is STARTED/PAUSED/BREAKDOWN and tracking_status is

ACTIVE or STALE. The UI displays last-known time and accuracy. Tracking ends at trip completion and never records unrelated personal journeys.

Owner self-drive requires approved Driver role, valid licence/documents, permission for the vehicle and no schedule conflict. External assignment remains pending until the Driver accepts.

Indexes: driver/status/start, owner/status/start, vehicle/scheduled time and active tracking partial index.

4.15 payments

Purpose: append-only financial events linked to a booking.

Core columns:

• id UUID PK; payment_reference VARCHAR(30) UNIQUE; booking_id UUID FK bookings(id).

• original_payment_id UUID NULL FK payments(id); payee_owner_user_id UUID NULL FK users(id).

• transaction_type: PAYMENT, REALLOCATION, ADDITIONAL_CHARGE, PARTIAL_REFUND, FULL_REFUND, OWNER_PAYOUT.

• payment_method: CARD, BANK, MOBILE, CASH, SANDBOX; provider_name; provider_reference.

• idempotency_key VARCHAR(100) UNIQUE; amount NUMERIC(12,2); commission_amount; owner_net_amount; currency CHAR(3).

• status: INITIATED, AUTHORIZED, CAPTURED, ALLOCATED, TRANSFERRED, REFUND_PENDING, REFUNDED, OWNER_PAYABLE, OWNER_PAID, FAILED, VOIDED.

• authorized_at, captured_at, refunded_at, owner_payable_at, owner_paid_at; failure_code; failure_message.

Rules: never overwrite a captured financial event to represent a refund or transfer; append a related transaction. A replacement keeps payment attached to the Passenger journey and reallocates it to the final operating trip/Owner. Higher fare needs approved additional charge; lower fare creates partial refund. Owner becomes payable only after final trip completion and reconciliation less commission.

Production custody and payout must use a licensed payment/marketplace provider. The NBP uses provider sandbox states and Admin-assisted cross-Owner reconciliation; it does not create an unlicensed Travelly wallet.

4.16 vehicle_records

Purpose: consolidate vehicle maintenance, expenses, odometer input, personal use and operational availability without losing typed meaning.

Core columns:

• id UUID PK; vehicle_id UUID FK vehicles(id); trip_id UUID NULL FK trips(id); recorded_by_user_id UUID FK users(id).

• record_type: OIL_CHANGE, OIL_FILTER_CHANGE, TYRE_CHANGE, TYRE_ROTATION, WHEEL_ALIGNMENT, WHEEL_BALANCING, GENERAL_SERVICE, BRAKE_SERVICE, BATTERY_SERVICE, REPAIR, FUEL_PURCHASE, EV_CHARGING, PERSONAL_EXTRA_USE, ODOMETER_UPDATE, AVAILABILITY_BLOCK.

• recorded_at; odometer_km; start_at NULL; end_at NULL; start_odometer_km NULL; end_odometer_km NULL.

• amount NUMERIC(12,2); currency; quantity; unit; fuel_type; fuel_grade_product; charging_kwh.

• next_due_odometer_km NULL; genuine_calendar_due_date NULL; service_provider; receipt_url; notes; approval_status.

Availability blocking: AVAILABILITY_BLOCK uses start_at/end_at and a reason such as maintenance or Owner hold. Vehicle search rejects an overlap when existing.start_at < requested.end_at AND existing.end_at > requested.start_at. Active bookings/trips and blocks are all checked before showing or assigning a vehicle.

Energy rules: every fuel purchase records the actual product, litres and paid amount. Different grades may be used on different dates. Charging stores actual amount and optional measured kWh. Litres and kWh remain separate units. Historical costs are not recalculated using today's price. Purchase expense is not automatically exact trip consumption.

Power compatibility: BEV has charging records and no engine-oil reminder; PHEV may have fuel, external charging and applicable engine maintenance; ICE/MHEV/HEV use fuel and applicable engine service. Manufacturer/model rules determine intervals.

Maintenance estimate:

remaining_km = next_due_odometer_km − vehicles.current_odometer_km

average_daily_km = accepted recorded distance during rolling period ÷ covered days

estimated_due_date = today + ceiling(remaining_km ÷ average_daily_km)

The UI shows current km, next due km, remaining km, estimated due date, genuine calendar due date and the data period. When data is insufficient or average distance is zero, it shows 'Estimated date unavailable'. The earlier genuine distance/calendar limit determines due or overdue state. Off-platform distance is recorded only through an accepted manual entry and is never invented.

4.17 notifications

Purpose: delivery and response record for operational communication.

Core columns:

• id UUID PK; recipient_user_id UUID FK users(id); booking_id UUID NULL; scheduled_trip_id UUID NULL; trip_id UUID NULL.

• notification_type: BOOKING_REQUEST, BOOKING_CONFIRMED, DRIVER_ASSIGNMENT, RECONFIRMATION, PAYMENT, REFUND, REPLACEMENT, MAINTENANCE_DUE, DOCUMENT_EXPIRY, ROAD_ALERT, ADMIN_ACTION.

• channel: IN_APP, EMAIL, SMS_MESSAGE, AUTOMATED_IVR.

• title; body; action_required BOOLEAN; action_payload JSONB.

• status: QUEUED, SENT, DELIVERED, READ, RESPONDED, FAILED, EXPIRED.

• sent_at, delivered_at, read_at, response_code, response_value, responded_at, expires_at, retry_count.

Rules: automated IVR may ask the Passenger to press 1 to confirm or 2 to indicate they cannot travel. It is not conversational AI calling. No response alone never cancels a paid booking or releases a seat/vehicle; the applicable cancellation rule and authorised action must be recorded.

4.18 social_posts

Purpose: basic Travelly Reels/photo/review content and discovery links.

Core columns:

• id UUID PK; author_user_id UUID FK users(id); post_type: REEL, PHOTO, REVIEW.

• media_url; thumbnail_url; caption; hashtags JSONB; location_name; latitude; longitude.

• linked_vehicle_id UUID NULL; linked_route_id UUID NULL; linked_scheduled_trip_id UUID NULL.

• visibility: PUBLIC, FOLLOWERS, PRIVATE; moderation_status: PENDING, APPROVED, REJECTED, HIDDEN.

• published_at; duration_seconds; denormalized_like_count, comment_count, share_count for fast display.

Media bytes stay outside PostgreSQL. Linking a post to a verified vehicle, route or departure enables the Passenger to move from a Reel to real search/booking. Counts are cache fields and are reconciled from social_interactions.

4.19 social_interactions

Purpose: typed social actions without separate table per interaction.

Core columns:

• id UUID PK; actor_user_id UUID FK users(id); post_id UUID NULL FK social_posts(id); target_user_id UUID NULL FK users(id).

• interaction_type: LIKE, COMMENT, SAVE, SHARE, FOLLOW, REPORT, RATING.

• comment_text NULL; rating SMALLINT NULL; report_reason NULL; interaction_status: ACTIVE, REMOVED, UNDER_REVIEW, RESOLVED.

• parent_interaction_id UUID NULL FK social_interactions(id) for comment replies; interaction_metadata JSONB.

Rules: target shape depends on type. LIKE/COMMENT/SAVE/SHARE/REPORT/RATING require post_id; FOLLOW requires target_user_id. Rating is 1–5. Partial unique indexes prevent duplicate active LIKE, SAVE and FOLLOW rows.

4.20 road_alerts

Purpose: temporary, route-aware alerts reported during an active trip.

Core columns:

• id UUID PK; reporter_user_id UUID FK users(id); trip_id UUID NULL FK trips(id).

• alert_type: POLICE_PRESENCE, SPEED_CHECK, ACCIDENT, ROADBLOCK, ROAD_HAZARD.

• latitude, longitude; travel_direction_degrees NULL; description; source_input: BUTTON, VOICE.

• status: ACTIVE, UNCONFIRMED, VERIFIED, EXPIRED, REJECTED, REMOVED.

• confidence_score; confirmation_count; denial_count; reported_at; verified_at; expires_at.

• moderated_by_user_id UUID NULL; moderation_reason.

Rules: button or voice input may create an alert. Voice AI only classifies the statement into a supported alert and requests clarification when uncertain. Eligible users receive a warning when approaching the relevant road/direction, approximately 1 km beforehand when route and GPS accuracy permit. Alerts expire automatically and are moderated. The feature is framed as lawful road-safety information and must not encourage illegal driving.

Indexes: status/expiry, trip_id, reporter and PostGIS GiST point index; without PostGIS use bounded latitude/longitude indexes plus application distance calculation.

5. Unified relationship map

The following is the authoritative single relationship model. Cardinality symbols are written in text so the design remains readable in Google Docs.

• users 1 — N user_roles

• users 1 — N documents; vehicles 1 — N documents, with exactly one owner target per document

• users (Owner) 1 — N vehicles

• vehicles N — M service_types through vehicle_services

• routes 1 — N route_stops

• routes 1 — N scheduled_trips

• vehicles 1 — N scheduled_trips as original/current operating vehicle

• vehicles 1 — N vehicle_seats

• scheduled_trips 1 — N scheduled_trip_seats; vehicle_seats 1 — N scheduled_trip_seats

• users (Passenger) 1 — N bookings

• bookings 1 — N booking_travellers

• scheduled_trip_seats 0..1 — 1 active booking_traveller

• bookings 0..1 — 1 trips for private hire/rental; scheduled_trips 1 — N trips only where operational splitting is later required, but NBP normally creates one operating trip

• users (Driver) 1 — N trips; vehicles 1 — N trips

• bookings 1 — N payments

• vehicles 1 — N vehicle_records; trips 0..1 — N vehicle_records

• users 1 — N notifications

• users 1 — N social_posts

• social_posts 1 — N social_interactions; users 1 — N social_interactions

• users 1 — N road_alerts; trips 0..1 — N road_alerts

Required delete behaviour: users, vehicles, bookings, trips, scheduled trips and payments use RESTRICT or archival status once referenced. Draft-only child records may cascade where data has no legal or audit value. Route stops and seat layouts cannot be deleted after a published departure references them; they become inactive.

6. Core workflow and transaction rules

6.1 Account and roles

Registration creates one users row and an approved Passenger role. Owner/Driver applications add user_roles rows and documents. Approval updates the relevant role only. A suspended Driver role does not automatically remove Passenger access, but active assignments are blocked.

6.2 Private hire and rental

Passenger search filters active service_types, vehicle_services, approved vehicles, document validity, capacity/luggage and availability overlap. A request creates a booking. The Owner may decline, accept and self-drive, or accept and assign a Driver. Self-drive creates/updates trips with the Owner as driver after eligibility checks. External assignment becomes operational only after Driver acceptance. Basic rental uses the same booking/payment/availability flow and applies rental document requirements from the service snapshot.

6.3 Ticket booking and 2D seats

Publishing a scheduled trip snapshots route, vehicle, fare, assurance policy, deadline and replacement limits, then creates one scheduled_trip_seats row per active seat. Checkout locks requested rows, changes them to HELD with an expiry, creates booking/traveller/payment attempts and confirms seats only after the authorised payment outcome. Failure rolls back or releases the hold. A scheduled job releases expired holds safely.

6.4 Travelly Protected fulfilment

Passenger sees Fixed Departure or Travelly Protected before payment. Minimum seat count is an internal Owner protection. If a protected departure misses its minimum, the Owner still fulfils the

journey using the original vehicle, an eligible own vehicle or an eligible replacement. The system may shortlist candidates; the other Owner must accept and Admin verifies the cross-Owner move and payout manually in the NBP. An equivalent replacement is notified. A material change is offered and requires explicit Passenger acceptance. If no acceptable fulfilment exists after permitted disruption handling, refund/support follows the accepted policy.

6.5 Vehicle availability

Search, booking confirmation, Driver assignment and departure publication all repeat the overlap check inside a transaction. Blocking states include HELD, CONFIRMED, DRIVER_ASSIGNMENT_PENDING, READY and IN_PROGRESS, plus active AVAILABILITY_BLOCK records. Advance and multi-day bookings block the complete [start_at, end_at) range. Database exclusion constraints using a generated tstzrange may be introduced for vehicle/time conflicts when the implementation can enforce only relevant active statuses; otherwise use transaction locks plus indexed overlap queries.

6.6 Payment lifecycle

Payment creation uses a unique idempotency key. Online Ticket booking requires the full payable amount before a seat becomes BOOKED. Booking confirmation and captured payment are reconciled explicitly; webhook retries cannot duplicate a charge. Refund, reallocation, additional charge and Owner payout are new payments rows linked through original_payment_id. In the NBP, cash is allowed only for enabled private-hire/basic-rental services. The Owner chooses start or end collection timing, and the Passenger plus collecting Driver or self-driving Owner must both confirm collection. Ticket cash/manual-sale handling is Phase 2. Owner payout occurs only after completion/reconciliation.

6.7 Trip execution and live location

A trip cannot become READY without an eligible vehicle and accepted operator. Start records odometer and activates tracking; periodic updates replace the current location fields on trips. Completion records end odometer, distance and tracking end, updates vehicles.current_odometer_km, evaluates maintenance reminders and unlocks final payment reconciliation. Full point-by-point history is intentionally deferred.

6.8 AI search

No AI conversation table is required. Text or voice input is processed statelessly to extract origin, destination, date/time, passenger count, service intent, explicit vehicle/luggage/power preference and trip-plan request. The backend validates the extracted values and queries actual eligible inventory. The model never invents a vehicle, departure, price or preference. Low-confidence fields are confirmed in the normal search form.

6.9 Social and road alerts

The basic vertical feed queries approved social_posts and interaction counts. Linked posts open verified vehicle/route/departure results. A road alert may be created only by the assigned active Driver or an eligible self-driving Owner on an active trip. Passengers may receive and submit feedback on alerts but cannot create an authoritative alert. The alert is shown only when active, geographically relevant, directionally relevant where available and not expired.

7. Status separation

The application must not use one generic status across all domains.

• Account/role status controls identity and permission.

• Document/vehicle verification status controls legal eligibility.

• Booking status controls the Passenger commercial request.

• Seat status controls inventory.

• Scheduled-trip status controls a published departure.

• Trip assignment/status controls actual operation.

• Payment status controls money movement.

• Notification status controls delivery/response.

• Moderation status controls social and alert visibility.

Each service implements an explicit allowed-transition map. Direct SQL updates that skip transition rules are prohibited outside controlled migrations and audited Admin repair.

.1 Explicit State-Transition Matrix

1 Explicit State-Transition Matrix

Explicit State-Transition Matrix

7.1 Explicit State-Transition Matrix

The backend enforces the following domain-specific paths. Branch-specific transition maps are version-controlled and verified by API and concurrency tests; a database value alone never authorizes the action.

Domain	Allowed NBP transition path	Mandatory guard
Role	REQUESTED → UNDER_REVIEW → APPROVED / REJECTED; APPROVED → SUSPENDED	Admin-only approval; role eligibility stays separate from account identity.
Document	DRAFT → SUBMITTED → UNDER_REVIEW → APPROVED / REJECTED / NEEDS_CORRECTION; APPROVED → EXPIRED / SUPERSEDED	Keep immutable versions and reviewer evidence.
Vehicle	DRAFT → SUBMITTED → APPROVED / REJECTED; APPROVED → SUSPENDED; only APPROVED vehicles may be published	Owner cannot self-approve; required documents must be valid.
Scheduled Ticket departure	DRAFT → PUBLISHED → BOOKING_OPEN → DECISION_REQUIRED / COMMITTED → READY → IN_PROGRESS → COMPLETED; policy may produce CANCELLED / DISRUPTED	Minimum-seat deadline, Travelly Protected replacement and Passenger-protection checks.
Seat inventory	AVAILABLE → HELD → BOOKED; HELD → AVAILABLE on expiry/payment failure; authorized BLOCKED state	Atomic row lock or conditional update; never double-book.
Booking	Service-specific DRAFT / HOLDING → REQUESTED or CONFIRMED → OWNER_ACCEPTED / DRIVER_ASSIGNMENT_PENDING → READY → IN_PROGRESS → COMPLETED; cancellation/refund states remain separate	Backend service guard, vehicle/time overlap check and payment rule.
Trip assignment / operation	PENDING → ACCEPTED / DECLINED / REASSIGN_REQUIRED; PREPARING → READY → STARTED → PAUSED / BREAKDOWN → COMPLETED / CANCELLED	Operator eligibility, document validity and calendar conflict checks.
Payment	INITIATED → AUTHORIZED → CAPTURED → ALLOCATED → OWNER_PAYABLE → OWNER_PAID; VOID / FAILED / REFUND states append to the history	Idempotency key, authenticated gateway event and reconciliation.
Notification	QUEUED → SENT → DELIVERED → READ / RESPONDED; FAILED / EXPIRED follow retry policy	No response is never treated as a decline.
Moderation / road alert	PENDING / ACTIVE → APPROVED / VERIFIED or REJECTED / HIDDEN / REMOVED / EXPIRED	Admin/system rules; road reporter must be assigned active Driver or eligible self-driving Owner.

8. Database constraints and concurrency

• Use foreign keys for every declared relationship and CHECK constraints for amounts, counts, coordinates, dates and target shapes.

• Use transaction boundaries for seat hold/confirmation, vehicle availability confirmation, Driver assignment acceptance, trip completion and payment reconciliation.

• Use SELECT … FOR UPDATE or atomic UPDATE … WHERE status = expected for contested rows.

• Use UNIQUE idempotency keys for booking creation and payment/webhook processing.

• Use optimistic version checks for Owner/Admin edits to vehicles, services, departures and bookings.

• Use partial unique indexes for one active seat assignment, one active like/save/follow and applicable active references.

• Use a scheduled worker with retry-safe queries for expired seat holds, document expiry, maintenance reminders, decision deadlines, reconfirmation and road-alert expiry.

• Do not rely only on frontend validation. Every compatibility, capacity, role and ownership rule is enforced again in the backend transaction.

9. Recommended indexes

• users(email), users(phone_e164), users(account_status).

• user_roles(user_id, role_code) UNIQUE and (role_code, approval_status).

• documents(user_id, document_type, status), documents(vehicle_id, document_type, status), documents(expiry_date, status).

• vehicles(owner_user_id), vehicles(registration_number) UNIQUE, vehicles(verification_status, is_published, availability_status).

• vehicle_services(vehicle_id, service_type_id) UNIQUE and (service_type_id, is_enabled).

• routes(origin_name, destination_name, status); route_stops(route_id, stop_order) UNIQUE.

• scheduled_trips(route_id, departure_at, status), (current_vehicle_id, departure_at), (decision_deadline, status).

• vehicle_seats(vehicle_id, layout_version, seat_code) UNIQUE.

• scheduled_trip_seats(scheduled_trip_id, vehicle_seat_id) UNIQUE, (scheduled_trip_id, seat_status), (hold_expires_at) WHERE seat_status='HELD'.

• bookings(passenger_user_id, created_at DESC), (owner_user_id, status, start_at), (vehicle_id, start_at, end_at).

• booking_travellers(booking_id), partial unique active scheduled_trip_seat_id.

• trips(driver_user_id, status, scheduled_start_at), (vehicle_id, scheduled_start_at), partial active tracking index.

• payments(booking_id, created_at), provider_reference, idempotency_key UNIQUE.

• vehicle_records(vehicle_id, record_type, recorded_at DESC), (vehicle_id, start_at, end_at) for availability blocks.

• notifications(recipient_user_id, status, created_at DESC), (expires_at, status).

• social_posts(moderation_status, published_at DESC); social_interactions(post_id, interaction_type).

• road_alerts(status, expires_at) plus GiST geospatial point index when PostGIS is enabled.

10. Security, privacy and retention

• Encrypt network traffic and storage; secrets and document access tokens remain outside ordinary application tables.

• Hash passwords with Argon2id or the approved framework default. Never log passwords, payment secrets or document URLs.

• Owner sees only owned fleet and assigned operations. Driver sees assigned vehicle/trip data. Passenger sees personal bookings and public marketplace data. Admin access is permission-controlled.

• Signed URLs for private documents expire quickly. Public vehicle/social images use separate public-safe assets.

• Live location is collected only during an active Travelly trip with visible tracking state and is replaced by last-known current location in the NBP.

• Retain payment, booking and completed-trip records according to legal/accounting policy. Personal data deletion requests anonymise what may legally be removed while preserving required transaction evidence.

• Road alerts and temporary seat holds have short expiry. Social removal hides public content while preserving moderation evidence where required.

11. Seed data and migration order

Recommended migration order:

1. PostgreSQL extensions: pgcrypto/uuid support, CITEXT and PostGIS if approved.

2. users, user_roles.

3. vehicles, documents.

4. service_types, vehicle_services.

5. routes, route_stops.

6. vehicle_seats, scheduled_trips.

7. bookings, scheduled_trip_seats, booking_travellers. Circular booking/seat foreign keys are added after both tables exist.

8. trips, payments, vehicle_records, notifications.

9. social_posts, social_interactions, road_alerts.

10. indexes, partial indexes, CHECK constraints, triggers/jobs and seed data.

Minimum seeds include roles, service types, supported document codes, one Jaffna–Colombo route with ordered stops, demo vehicle/seat layout, notification types and alert types. Enumerations may be PostgreSQL ENUMs or validated VARCHAR values; choose one approach consistently. Business values expected to change belong in service_types/settings rather than database ENUMs.

12. Validation and acceptance tests

• One account gains Passenger, Owner and Driver roles without duplicate users.

• Unapproved Owner/Driver or expired mandatory document cannot publish/operate.

• Owner self-drive succeeds only with approved Driver eligibility and no schedule conflict.

• Two concurrent checkouts cannot book the same scheduled seat.

• Expired seat hold returns to AVAILABLE without cancelling a confirmed seat.

• Advance and multi-day vehicle bookings block overlapping searches but allow adjacent non-overlapping time ranges.

• Travelly Protected booking remains a confirmed obligation; minimum-seat failure triggers an approved fulfilment decision rather than silent cancellation.

• Replacement preserves original/current departure and vehicle, Passenger acceptance where material, payment allocation and final Owner payable.

• Duplicate payment/webhook retry does not create a second charge.

• Trip start/finish updates operational status, odometer and maintenance calculation correctly.

• BEV rejects fuel/oil-only records; applicable hybrids accept correct fuel/charging combinations.

• No-response to IVR does not release inventory.

• Owner sees live/last-known location only for the active assigned trip.

• AI text/voice search returns only database-backed eligible inventory.

• Expired or rejected road alerts are not shown; reported social content enters moderation.

• All twenty tables can be created and seeded from an empty database without manual edits.

13. One-month implementation boundary

This 20-table design contains the complete NBP core. It is sufficient for a successful working application when the API, status-transition rules, transactions, permissions, background jobs and indexes described here are implemented. More users do not require more tables; scale depends mainly on indexes, query design, caching, media storage, connection pooling and infrastructure.

New ideas during the build receive one decision: fit them into an existing NBP field/flow; replace or merge an existing design when required for integrity; or place them in Phase 2. The database table count remains exactly 20 for the one-month release.

14. Final implementation decision

Approved NBP database count: 20 application tables.

Implementation authority: this document.

Old 149-table architecture: archived for future expansion and not to be used for the current build.

Proposal document and API specification are maintained separately and must remain aligned with this final schema.

Implementation next step: generate PostgreSQL migrations and backend services from this approved 20-table schema and the aligned endpoint-by-endpoint API specification.